COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithms | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | 09/08/2026 | **DATE SUBMITTED:** | 09/08/2026 |
| **NAME:** | Cabug, Jeffordniel H. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**HANDS-ON ACTIVITY NO. 7.1** **Sorting Algorithms Pt1**

## 1. PROCEDURE OUTPUT

**ILO A: Create C++ code for implementation of selection, bubble, insertion and merge sort.**

Code + Console Screenshot

**CODE:**

```
7     // array with 100 size
8     const int SIZE = 100;
9     int unsortedArr[SIZE];
10
11    // Random Values
12    std::srand(std::time(0));
13
14    // Generate 100 random values
15    for (int i = 0; i < SIZE; i++) {
16        unsortedArr[i] = std::rand() % 100 + 1;
17    }
18
19    // Display all elements
20    std::cout << "Unsorted Array: "<<std::endl;
21    display(unsortedArr, SIZE);
```

```
5   // FUNCTION VALUES SORT
6   void display(int arr[], int size) {
7       for (int i = 0; i < size; i++) {
8           std::cout << arr[i] << " ";
9       }
10      std::cout << std::endl;
11  }
12
```

**OUTPUT:**

```
Unsorted Array:
7 68 73 55 38 80 52 82 72 41 45 79 86 59 71 66 45 53 27 10 94 96 10 78 8 39 30 21 3 98 3
2 45 80 14 73 84 97 81 51 49 38 83 75 34 64 29 89 13 9 65 44 4 33 61 66 16 83 86 58 18 4
6 84 64 29 26 1 4 18 62 83 99 93 24 10 63 83 23 65 50 56 15 8 68 44 74 38 84 96 97 20 79
 62 30 17 43 57 81 79 72 41
```

| | |
|---|---|
| Observations | Constant **SIZE** with a value of 100, which means the array will contain 100 elements. The int **unsortedArr[SIZE]** statement creates an integer array named unsortedArr with 100 spaces. The **srand(std::time(0))** statement initializes the random-number generator using the current time so that different random values are produced each time the program runs. The for loop then goes through each position of the array, from 0 to 99, and **rand() % 100 + 1** generates a random integer between 1 and 100, which is stored in the array. |

Table 7-1. Array of Values for Sort Algorithm Testing

| | |
|---|---|
| Code + Console Screenshot | **CODE:**<br>`// BUBBLE SORT`<br>`template <typename T>`<br>`void bubblesort(T arr[], int SIZE) {`<br>`  for (int i = 0; i < SIZE; i++) {`<br>`  for (int j = i +1; j < SIZE; j++){`<br>`  if (arr[i] > arr[j]){`<br>`  std::swap(arr[i], arr[j]);`<br>`  }`<br>`  }`<br>`  }`<br>`}`<br>`std::cout << "Bubble sort: "<<std::endl;`<br>`bubblesort(unsortedArr, SIZE);`<br>`display(unsortedArr, SIZE);`<br>**OUTPUT:**<br>`Bubble sort:`<br>`1 3 4 4 7 8 8 9 10 10 10 13 14 15 16 17 18 18 20 21 23 24 26 27 29 29 30 30 32 33 34 38 38 38 39 41 41 43 44 44 45 45 45 46 49 50 51 52 53 55 56 57 58 59 61 62 62 63 64 64 65 65 66 66 68 68 71 72 72 73 73 74 75 78 79 79 79 80 80 81 81 82 83 83 83 83 84 84 84 86 86 89 93 94 96 96 97 97 98 99` |
| Observations | Using the Bubble Sort to arrange the array in ascending order. It compares each element with the values after it and swaps them when they are in the wrong order. After sorting, the program displays the resulting array from smallest to largest. The void **bubblesort(T arr[], int SIZE)** statement creates a function that receives the array and its size. The outer for loop controls the sorting process, while the inner for loop compares each element with the elements after it. The if **(arr[i] > arr[j])** statement checks if the values are in the wrong order, and **swap(arr[i], arr[j])** exchanges their positions when necessary. |

Table 7-2. Bubble Sort Technique

| | |
|---|---|
| Code + Console Screenshot | **CODE:**<br>`// SELECTION SORT`<br>`template <typename T>`<br>`void selectionsort(T arr[], int arrSIZE) {`<br>`  int min, temp;`<br><br>`  for (int i = 0; i < arrSIZE; i++) {`<br>`  min = routine_smallest(arr, i , arrSIZE);`<br>`  temp = arr[i];`<br>`  arr[i] = arr[min];`<br>`  arr[min] = temp;`<br>`  //std::swap(arr[i], arr[min]);`<br><br><br>`  }`<br><br>`std::cout << "Selection sort: "<<std::endl;`<br>`bubblesort(unsortedArr, SIZE);`<br>`display(unsortedArr, SIZE);`<br><br>**OUTPUT:**<br>`Selection sort:`<br>`1 3 4 4 7 8 8 9 10 10 10 13 14 15 16 17 18 18 20 21 23 24 26 27 29 29 30 30 32 33 34 38 38 38 39 41 41 43 44 44 45 45 45 46 49 50 51 52 53 55 56 57 58 59 61 62 62 63 64 64 65 6 5 66 66 68 68 71 72 72 73 73 74 75 78 79 79 79 80 80 81 81 82 83 83 83 83 84 84 84 86 86 89 93 94 96 96 97 97 98 99` |
| Observations | The selectionsort() function receives the array and its size, while the variables min and temp are used to identify and temporarily store values during the sorting process. The for loop goes through each position of the array and uses routine_smallest() to find the smallest element in the unsorted portion. The current element is then stored in temp, while the smallest element is moved to the current position of the array. The value originally at the current position is placed where the smallest element was found, completing the swap. The loop continues until all elements have been arranged. |

Table 7-3. Selection Sort Algorithm

| | |
|---|---|
| Code + Console Screenshot | **CODE:**<br>`// INSERTION SORT`<br>`template <typename T>`<br>`void insertionsort(T arr[], int arrSIZE) {`<br>`  int k = 1, j;`<br>`  T temp;`<br>`  while(k < arrSIZE){`<br>`  temp = arr[k];`<br>`  j = k -1;`<br>`  while (j >= 0 && temp <= arr[j] ){`<br>`  arr[j+1] = arr[j];`<br>`  j--;`<br>`  }`<br>`  arr [j+1] = temp;`<br>`  k++;`<br>`  }`<br>`}`<br>`std::cout << "Insertion sort: "<<std::endl;`<br>`insertionsort(unsortedArr, SIZE);`<br>`display(unsortedArr, SIZE);`<br>**OUTPUT**:<br>`Insertion sort:`<br>`1 3 4 4 7 8 8 9 10 10 10 13 14 15 16 17 18 18 20 21 23 24 26 27 29 29 30 30 32 33 34 38 38 38 39 41 41 43 44 44 45 45 45 46 49 50 51 52 53 55 56 57 58 59 61 62 62 63 64 64 65 65 66 66 68 68 71 72 72 73 73 74 75 78 79 79 79 80 80 81 81 82 83 83 83 83 84 84 84 86 86 89 93 94 96 96 97 97 98 99` |
| Observations | The insertionsort() function receives the array and its size, while the variables k, j, and temp are used to control the sorting process and temporarily store values. The outer while loop processes each element of the array, storing the current element in temp and setting j to the position before it. The inner while loop compares the current element with the previous elements and shifts larger values one position to the right. Once the correct position is found, temp is inserted into the array, and k is increased to continue with the next element. |

Table 7-4. Insertion Sort Algorithm

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**ILO B: Solve given data sorting problems using appropriate basic sorting algorithms**

**CODE (MAIN):**

Supplementary > main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "insertionsort.h"

int main() {
    const int SIZE = 100;
    int A[SIZE];
    int count[6] = {0};

    // GENERATES 100 RANDOM VOTES
    std::srand(std::time(0));

    for (int i = 0; i < SIZE; i++) {
        A[i] = std::rand() % 5 + 1;
    }

    // UNSORTED ARRAY
    std::cout << "Unsorted: " << std::endl;

    for (int i = 0; i < SIZE; i++) {
        std::cout << A[i] << " ";

        if ((i + 1) % 20 == 0)
            std::cout << std::endl;
    }

    // INSERTION SORT
    insertionsort(A, SIZE);

    // SORTED ARRAY
    std::cout << "\nSorted: " << std::endl;

    for (int i = 0; i < SIZE; i++) {
        std::cout << A[i] << " ";

        if ((i + 1) % 20 == 0)
            std::cout << std::endl;
    }

    // MANUAL COUNT VOTING RESULT
    int manualCount[6] = {0};

    for (int i = 0; i < SIZE; i++) {
        manualCount[A[i]]++;
    }

    std::cout << "\nManual Count: " << std::endl;
```

```
49
50      for (int i = 1; i <= 5; i++) {
51          std::cout << "Vote " << i << ": "
52                    << manualCount[i] << std::endl;
53      }
54
55      // COUNTING VOTING RESULT
56      for (int i = 0; i < SIZE; i++) {
57          count[A[i]]++;
58      }
59
60      // WINNER RESULT
61      int winner = 1;
62
63      for (int i = 2; i <= 5; i++) {
64          if (count[i] > count[winner]) {
65              winner = i;
66          }
67      }
68
69      std::cout << "\nCANDIDATE VOTING RESULTS:" << std::endl;
70
71      std::cout << "Candidate 1 - Bo Dalton Capistrano: "
72                << count[1] << " votes" << std::endl;
73
74      std::cout << "Candidate 2 - Cornelius Raymon Agustin: "
75                << count[2] << " votes" << std::endl;
76
77      std::cout << "Candidate 3 - Deja Jayla Banaga: "
78                << count[3] << " votes" << std::endl;
79
80      std::cout << "Candidate 4 - Lalla Brielle Yabut: "
81                << count[4] << " votes" << std::endl;
82
83      std::cout << "Candidate 5 - Franklin Relano Castro: "
84                << count[5] << " votes" << std::endl;
85
86      std::cout << "The winner is ";
87
88      switch (winner) {
89          case 1:
90              std::cout << "Bo Dalton Capistrano";
91              break;
92
93          case 2:
94              std::cout << "Cornelius Raymon Agustín";
95              break;
96
```

```
97          case 3:
98              std::cout << "Deja Jayla Bañaga";
99              break;
100
101         case 4:
102             std::cout << "Lalla Brielle Yabut";
103             break;
104
105         case 5:
106             std::cout << "Franklin Relano Castro";
107             break;
108     }
109
110     std::cout << " With " << count[winner] << " votes!" << std::endl;
111
112     return 0;
113 }
```

**(INSERTIONSORT_H)**

Supplementary > C insertionsort.h > ...

```
1   #ifndef INSERTIONSORT_H
2   #define INSERTIONSORT_H
3
4   void insertionsort(int A[], int size) {
5
6       for (int i = 1; i < size; i++) {
7           int key = A[i];
8           int j = i - 1;
9
10          while (j >= 0 && A[j] > key) {
11              A[j + 1] = A[j];
12              j--;
13          }
14
15          A[j + 1] = key;
16      }
17  }
18
19  #endif
```

**OUTPUT:**

```
Unsorted:
1 5 3 2 4 5 2 3 3 4 2 4 3 4 2 4 5 5 1 2
4 1 4 4 1 5 3 5 3 2 5 3 2 3 2 2 3 5 1 2
1 5 3 1 2 5 5 5 3 3 1 1 5 2 3 3 3 1 4 3
4 3 4 1 4 1 4 2 3 2 2 5 2 5 2 1 1 1 2 1
5 4 5 1 4 2 4 2 4 2 4 2 4 1 1 2 3 2 1 4

Sorted:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Manual Count:
Vote 1: 20
Vote 2: 24
Vote 3: 19
Vote 4: 20
Vote 5: 17

CANDIDATE VOTING RESULTS:
Candidate 1 - Bo Dalton Capistrano: 20 votes
Candidate 2 - Cornelius Raymon Agustin: 24 votes
Candidate 3 - Deja Jayla Banaga: 19 votes
Candidate 4 - Lalla Brielle Yabut: 20 votes
Candidate 5 - Franklin Relano Castro: 17 votes
The winner is Cornelius Raymon Agust├¡n With 24 votes!
```

| Output Console Showing Sorted Array | Manual Count | Count Result of Algorithm |
|---|---|---|
| Sorted:<br>1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1<br>2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2<br>2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3<br>3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4<br>4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 | Manual Count:<br>Vote 1: 20<br>Vote 2: 24<br>Vote 3: 19<br>Vote 4: 20<br>Vote 5: 17 | CANDIDATE VOTING RESULTS:<br>Candidate 1 - Bo Dalton Capistrano: 20 votes<br>Candidate 2 - Cornelius Raymon Agustin: 24 votes<br>Candidate 3 - Deja Jayla Banaga: 19 votes<br>Candidate 4 - Lalla Brielle Yabut: 20 votes<br>Candidate 5 - Franklin Relano Castro: 17 votes |

| PSEUDO CODE |
|---|
| START<br><br>SET INT SIZE = 100 |

```
Int array A[100]
Int count [6] and set all values to 0

FOR i = 0 TO 99
   A[i] = random number from 1 to 5
END FOR

DISPLAY "Unsorted:"
FOR i = 0 TO 99
   DISPLAY A[i]
   IF (i + 1) is divisible by 20
   END IF
END FOR

SORT A using INSERTION SORT

DISPLAY "Sorted:"
FOR i = 0 TO 99
   DISPLAY A[i]
   IF (i + 1) is divisible by 20
   END IF
END FOR

CREATE manualCount[6] and set all values to 0

FOR i = 0 TO 99
   manualCount[A[i]] = manualCount[A[i]] + 1
END FOR

DISPLAY "Manual Count:"
FOR i = 1 TO 5
   DISPLAY "Vote", i, manualCount[i]
END FOR

FOR i = 0 TO 99
   count[A[i]] = count[A[i]] + 1
END FOR

SET INT winner = 1

FOR i = 2 TO 5
   IF count[i] > count[winner]
      winner = i
   END IF
END FOR

DISPLAY "CANDIDATE VOTING RESULTS:"

DISPLAY Candidate 1 and count[1]
DISPLAY Candidate 2 and count[2]
DISPLAY Candidate 3 and count[3]
DISPLAY Candidate 4 and count[4]
DISPLAY Candidate 5 and count[5]

IF winner = 1
   DISPLAY "Bo Dalton Capistrano"
ELSE IF winner = 2
   DISPLAY "Cornelius Raymon Agustin"
```

```
ELSE IF winner = 3
   DISPLAY "Deja Jayla Banaga"
ELSE IF winner = 4
   DISPLAY "Lalla Brielle Yabut"
ELSE IF winner = 5
   DISPLAY "Franklin Relano Castro"
END IF

DISPLAY number of votes received by the winner

END
```

**Question:** Was your developed vote counting algorithm effective? Why or why not?

- Yes, the developed vote counting algorithm was effective because it correctly sorted the votes and counted the votes for each candidate. Manual count and algorithm count have the same result. It can also recognize the candidate with the highest number of votes.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **CHATGPT GEEK FOR GEEKS**

*.,mnvnnm Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Explanation of Concepts, formatting**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **AI Use for explaining the concept of sorting algorithm.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.